大模型 RAG 经验面

来自： AiGC面试宝典

宁静致远

2024年01月28日 10:12

扫码
查看更

- RAG（Retrieval-Augmented Generation）面
  - 一、LLMs 已经具备了较强能力了，存在哪些不足点?
  - 二、什么是 RAG?
    - 2.1 R：检索器模块
      - 2.1.1 如何获得准确的语义表示?
      - 2.1.2 如何协调查询和文档的语义空间?
      - 2.1.3 如何对齐检索模型的输出和大语言模型的偏好?
    - 2.2 G：生成器模块
      - 2.2.1 生成器介绍
      - 2.2.2 如何通过后检索处理提升检索结果?
      - 2.2.3 如何优化生成器应对输入数据?
  - 三、使用 RAG 的好处?
  - 四、RAG V.S. SFT
  - 五、介绍一下 RAG 典型实现方法?
    - 5.1 如何 构建 数据索引?
    - 5.2 如何 对数据进行 检索（Retrieval）?
    - 5.3 对于 检索到的文本，如果生成正确回复?
  - 六、介绍一下 RAG 典型案例?
    - 6.1 ChatPDF 及其 复刻版
    - 6.2 Baichuan
    - 6.3 Multi-modal retrieval-based LMs
  - 七、RAG 存在什么问题?
  - 致谢

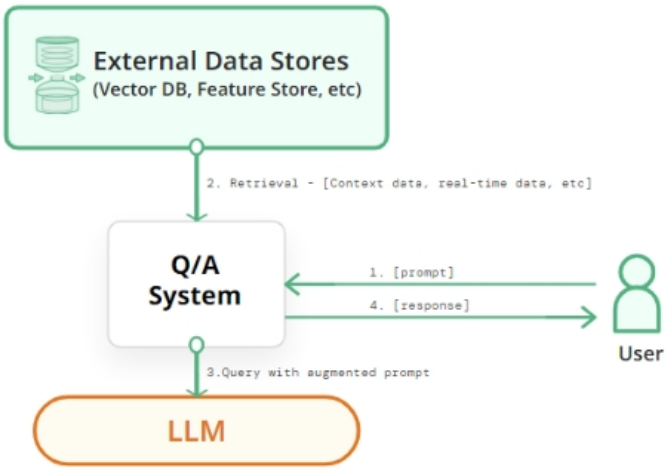
一、LLMs 已经具备了较强能力了，存在哪些不足点?

在 LLM 已经具备了较强能力的基础上，仍然存在以下问题：

- **幻觉问题**：LLM 文本生成的底层原理是基于概率的 token by token 的形式，因此会不可避免地产生“一本正经的胡说八道”的情况；
- **时效性问题**：LLM 的规模越大，大模型训练的成本越高，周期也就越长。那么具有时效性的数据也就无法参与训练，所以也就无法直接回答时效性相关的问题，例如“帮我推荐几部热映的电影？”；
- **数据安全问题**：通用的 LLM 没有企业内部数据和用户数据，那么企业想要在保证安全的前提下使用 LLM，最好的方式就是把数据全部放在本地，企业数据的业务计算全部在本地完成。而在线的大模型仅仅完成一个归纳的功能；

二、什么是 RAG?

RAG（Retrieval Augmented Generation, 检索增强生成），即 LLM 在回答问题或生成文本时，先会从大量文档中检索出相关的信息，然后基于这些信息生成回答或文本，从而提高预测质量。



RAG for LLMs

2.1 R：检索器模块

在 RAG技术中，“R”代表检索，其作用是从大量知识库中检索出最相关的前 k 个文档。然而，构建一个高质量的检索器是一项挑战。研究探讨了三个关键问题：

2.1.1 如何获得准确的语义表示?

在 RAG 中，语义空间指的是查询和文档被映射的多维空间。以下是两种构建准确语义空间的方法。

1. 块优化

处理外部文档的第一步是分块，以获得更细致的特征。接着，这些文档块被嵌入。

选择分块策略时，需要考虑被索引内容的特点、使用的嵌入模型及其最适块大小、用户查询的预期长度和复杂度、以及检索结果在特定应用中的使用方式。实际上，准确的查询结果是通过灵活应用多种分块策略来实现的，并没有最佳策略，只有最适合的策略。

1. 微调嵌入模型

在确定了 Chunk 的适当大小之后，我们需要通过一个嵌入模型将 Chunk 和查询嵌入到语义空间中。如今，一些出色的嵌入模型已经问世，例如 UAE、Voyage、BGE等，它们在大规模语料库上预训练过

2.1.2 如何协调查询和文档的语义空间?

在 RAG 应用中，有些检索器用同一个嵌入模型来处理查询和文档，而有些则使用两个不同的模型。此外，用户的原始查询可能表达不清晰或缺少必要的语义信息。因此，协调用户的查询与文档的语义空间显得尤为重要。研究介绍了两种关键技术：

1. 查询重写

一种直接的方式是对查询进行重写。

可以利用大语言模型的能力生成一个指导性的伪文档，然后将原始查询与这个伪文档结合，形成一个新的查询。

也可以通过文本标识符来建立查询向量，利用这些标识符生成一个相关但可能并不存在的“假想”文档，它的目的是捕捉到相关的模式。

此外，多查询检索方法让大语言模型能够同时产生多个搜索查询。这些查询可以同时运行，它们的结果一起被处理，特别适用于那些需要多个小问题共同解决的复杂问题。

1. 嵌入变换

在 Liu 于 2023 年提出的 LlamaIndex 中，研究者们通过在查询编码器后加入一个特殊的适配器，并对其进行微调，从而优化查询的嵌入表示，使之更适合特定的任务。

Li 团队在 2023 年提出的 SANTA 方法，就是为了让检索系统能够理解并处理结构化的信息。他们提出了两种预训练方法：一是利用结构化与非结构化数据之间的自然对应关系进行对比学习；二是采用了一种围绕实体设计的掩码策略，让语言模型来预测和填补这些被掩盖的实体信息。

2.1.3 如何对齐检索模型的输出和大语言模型的偏好?

在 RAG流水线中，即使采用了上述技术来提高检索模型的命中率，仍可能无法改善 RAG 的最终效果，因为检索到的文档可能不符合大语言模型的需求。

因此，研究介绍了如下方法：

大语言模型的监督训练：REPLUG使用检索模型和大语言模型计算检索到的文档的概率分布，然后通过计算 KL 散度进行监督训练。

这种简单而有效的训练方法利用大语言模型作为监督信号，提高了检索模型的性能，消除了特定的交叉注意力机制的需求。

此外，也有一些方法选择在检索模型上外部附加适配器来实现对齐，这是因为微调嵌入模型可能面临一些挑战，比如使用 API 实现嵌入功能或计算资源不足等。因此，一些方法选择在检索模型上外部附加适配器来实现对齐。

除此之外，PKG通过指令微调将知识注入到白盒模型中，并直接替换检索模块，用于根据查询直接输出相关文档。

2.2 G：生成器模块

2.2.1 生成器介绍

- 介绍：在 RAG 系统中，生成组件是核心部分之一
- 作用：将检索到的信息转化为自然流畅的文本。在 RAG 中，生成组件的输入不仅包括传统的上下文信息，还有通过检索器得到的相关文本片段。这使得生成组件能够更深入地理解问题背后的上下文，并产生更加信息丰富的回答。此外，生成组件还会根据检索到的文本来指导内容的生成，确保生成的内容与检索到的信息保持一致。

正是因为输入数据的多样性，我们针对生成阶段进行了一系列的有针对性工作，以便更好地适应来自查询和文档的输入数据。

2.2.2 如何通过后检索处理提升检索结果?

- 介绍：后检索处理指的是，在通过检索器从大型文档数据库中检索到相关信息后，对这些信息进行进一步的处理、过滤或优化。
- 主要目的：提高检索结果的质量，更好地满足用户需求或为后续任务做准备。
- 后检索处理策略：包括信息压缩和结果的重新排序。

2.2.3 如何优化生成器应对输入数据?

- 生成器工作：负责将检索到的信息转化为相关文本，形成模型的最终输出。
- 其优化目的：在于确保生成文本既流畅又能有效利用检索文档，更好地回应用户的查询。

RAG 的输入不仅包括查询，还涵盖了检索器找到的多种文档（无论是结构化还是非结构化）。一般在将输入提供给微调过的模型之前，需要对检索器找到的文档进行后续处理。
值得注意的是，RAG 中对生成器的微调方式与大语言模型的普通微调方法大体相同，包括有通用优化过程以及运用对比学习等。

三、使用 RAG 的好处?

RAG 方法使得开发者不必为每一个特定的任务重新训练整个大模型，只需要外挂上知识库，即可为模型提供额外的信息输入，提高其回答的准确性。RAG模型尤其适合知识密集型的任务。

- **可扩展性 (Scalability)**：减少模型大小和训练成本，并允许轻松扩展知识
- **准确性 (Accuracy)**：通过引用信息来源，用户可以核实答案的准确性，这增强了人们对模型输出结果的信任。
- **可控性 (Controllability)**：允许更新或定制知识
- **可解释性 (Interpretability)**：检索到的项目作为模型预测中来源的参考
- **多功能性 (Versatility)**：RAG 可以针对多种任务进行微调和定制，包括QA、文本摘要、对话系统等；
- **及时性**：使用检索技术能识别到最新的信息，这使 RAG 在保持回答的及时性和准确性方面，相较于只依赖训练数据的传统语言模型有明显优势。
- **定制性**：通过索引与特定领域相关的文本语料库，RAG 能够为不同领域提供专业的知识支持。
- **安全性**：RAG 通过数据库中设置的角色和安全控制，实现了对数据使用的更好控制。相比之下，经过微调的模型在管理数据访问权限方面可能不够明确。

四、RAG V.S. SFT

实际上，对于 LLM 存在的上述问题，SFT 是一个最常见最基本的解决办法，也是 LLM 实现应用的基础步骤。那么有必要在多个维度上比较一下两种方法：